

Higher order functions

Trịnh Tấn Đạt

Đại Học Sài Gòn

trinhtandat@sgu.edu.vn

<http://sites.google.com/site/ttdat88>

Nội dung

- Docstring của hàm
- Lambda Expressions
- Higher-order functions
- Function closure
- Wrapper
- Decorator

Mục tiêu học tập

- Viết hàm Python có docstring rõ ràng.
- Giải thích được vì sao hàm là một đối tượng trong Python.
- Dùng lambda cho các biểu thức ngắn.
- Truyền hàm vào hàm khác và trả về hàm.
- Giải thích closure, wrapper và decorator.
- Viết decorator đơn giản để ghi log hoặc đo thời gian thực thi.
- Áp dụng các kỹ thuật trên cho dữ liệu dạng danh sách/dictionary.

Hàm là đối tượng

- Trong Python, hàm không chỉ là khối lệnh được gọi để thực hiện công việc. Hàm còn là một đối tượng có thể:
 - Gán vào biến.
 - Lưu trong list hoặc dictionary.
 - Truyền vào hàm khác như một tham số.
 - Được trả về từ một hàm khác.
 - Được bao bọc bởi một hàm khác.

Hàm là đối tượng

```
def greet(name):  
    return f"Hello, {name}!"
```

```
message_function = greet
```

```
print(message_function("Nam"))
```

message_function = greet <-- Chúng ta **không** gọi hàm.

Kết quả:

Hello, Nam!

Gọi hàm và truyền hàm

```
def square(x):  
    return x * x  
  
print(square(5))
```

Trong khi đó:

```
f = square  
print(f(5))
```

nghĩa là gán đối tượng hàm `square` vào biến `f`.

`square(5)` nghĩa là gọi hàm và nhận kết quả là 25.

Cú pháp	Ý nghĩa
<code>square</code>	Đối tượng hàm
<code>square(5)</code>	Gọi hàm với đối số 5
<code>f = square</code>	Gán hàm vào biến
<code>f(5)</code>	Gọi hàm thông qua biến <code>f</code>

Docstring là gì?

- Docstring là chuỗi mô tả được đặt ở vị trí đầu tiên trong thân của:
 - Module.
 - Hàm.
 - Class.
 - Method.
- Docstring giúp mô tả:
 - Mục đích của hàm.
 - Ý nghĩa tham số đầu vào.
 - Kiểu và ý nghĩa kết quả trả về.
 - Ngoại lệ có thể phát sinh.
 - Ví dụ sử dụng.

```
def add(a, b):  
    """Trả về tổng của hai số a và b."""  
    return a + b
```

Docstring và comment

Tiêu chí	Comment	Docstring
Ký hiệu	#	Chuỗi """ ... """
Mục đích	Giải thích từng đoạn code	Tài liệu hóa hàm/class/module
Truy cập khi chạy	Không	Có
Dùng với help()	Không	Có
Thuộc tính __doc__	Không	Có

```
def calculate_mean(values):  
    """Tính giá trị trung bình của một danh sách số."""  
    return sum(values) / len(values)  
  
print(calculate_mean.__doc__)
```

Mẫu docstring

```
def calculate_mean(values):  
    """  
    Tính giá trị trung bình của danh sách số.  
  
    Parameters  
    -----  
    values : list[float]  
        Danh sách các giá trị số.  
  
    Returns  
    -----  
    float  
        Giá trị trung bình của danh sách.  
  
    Raises  
    -----  
    ValueError  
        Nếu danh sách đầu vào rỗng.  
    """  
    if not values:  
        raise ValueError("Danh sách không được rỗng")  
  
    return sum(values) / len(values)
```

Lambda expression

- Lambda là hàm ẩn danh, thường dùng cho logic đơn giản trên một biểu thức.
- Cú pháp: `lambda tham_so: bieu_thuc`

```
square = lambda x: x * x  
print(square(5))
```

Tương đương:

```
def square(x):  
    return x * x  
print(square(5))
```

Khi nào nên dùng lambda?

- Nên dùng lambda khi
 - Hàm rất ngắn.
 - Hàm chỉ cần dùng một lần.
 - Hàm được truyền vào `sorted()`, `map()`, `filter()`.
 - Lambda giúp biểu đạt logic rõ ràng hơn.
- Không nên dùng lambda khi
 - Có nhiều bước xử lý.
 - Có nhiều điều kiện lồng nhau.
 - Cần docstring.
 - Cần tái sử dụng hàm ở nhiều nơi.
 - Tên hàm giúp code dễ hiểu hơn.

Lambda với sorted()

```
students = [  
    ("An", 8.2),  
    ("Binh", 7.5),  
    ("Chi", 9.1),  
    ("Dung", 8.7)  
]  
  
result = sorted(  
    students,  
    key=lambda student: student[1],  
    reverse=True  
)  
  
print(result)
```

Kết quả:

```
[  
    ("Chi", 9.1),  
    ("Dung", 8.7),  
    ("An", 8.2),  
    ("Binh", 7.5)  
]
```

Lambda với map() và filter()

```
pm25 = [12.5, 18.2, 22.1, 35.6, 41.3]

squared_values = list(map(lambda x: x ** 2, pm25))

high_values = list(filter(lambda x: x > 25, pm25))

print(squared_values)
print(high_values)
```

Hàm	Vai trò
map(function, iterable)	Biến đổi từng phần tử
filter(function, iterable)	Lọc phần tử thỏa điều kiện
list(...)	Chuyển iterator về list để hiển thị

Bài tập thực hành 1

1. Viết ba hàm có docstring đầy đủ:

```
square(n)  
is_even(n)  
celsius_to_fahrenheit(c)
```

Mỗi docstring cần mô tả:

- Chức năng.
- Tham số.
- Kết quả trả về.
- Kiểu dữ liệu đầu vào/đầu ra.

Bài tập thực hành 1

2. Cho dữ liệu:

```
temperatures = [28, 31, 35, 29, 37, 33]
```

Yêu cầu:

- Dùng `map()` và `lambda` đổi nhiệt độ C sang F.
- Dùng `filter()` lấy ngày có nhiệt độ lớn hơn 32.
- Dùng `sorted()` sắp xếp giảm dần.

Higher-order functions

- Higher-order function là hàm thỏa ít nhất một trong hai điều kiện:
 - Nhận một hoặc nhiều hàm khác làm đối số.
 - Trả về một hàm khác.

```
def apply_function(func, value):  
    return func(value)
```

```
def square(x):  
    return x * x
```

```
print(apply_function(square, 5))
```

Cấu trúc tổng quát

```
def higher_order_function(function, data):  
    result = function(data)  
    return result
```

Lợi ích:

- Tái sử dụng logic.
- Tách dữ liệu khỏi cách xử lý.
- Dễ mở rộng chức năng.
- Giảm sao chép code.

Dữ liệu đầu vào
↓
Hàm xử lý được truyền vào
↓
Higher-order function
↓
Kết quả đầu ra

Ví dụ: áp dụng hàm hai lần

```
def apply_twice(func, value):  
    return func(func(value))
```

```
def increment(x):  
    return x + 1
```

```
def square(x):  
    return x * x
```

```
print(apply_twice(increment, 3))  
print(apply_twice(square, 2))
```

`apply_twice(increment, 3)`

Tương đương

`increment(increment(3))`

3
↓ increment
4
↓ increment
5

2 → 4 → 16

Ví dụ: chọn kỹ thuật xử lý

```
def transform_values(values, func):  
    return [func(x) for x in values]  
  
def normalize_by_100(x):  
    return x / 100  
  
def square(x):  
    return x ** 2  
  
values = [10, 20, 30]  
  
print(transform_values(values, normalize_by_100))  
print(transform_values(values, square))
```

Ý tưởng

- Dữ liệu giữ nguyên cấu trúc.
- Hàm xử lý có thể thay đổi linh hoạt.
- Đây là tiền đề của pipeline xử lý dữ liệu.

Callback function

- Callback là hàm được truyền vào một hàm khác để được gọi ở một thời điểm phù hợp.

```
def process_data(values, callback):  
    result = [x * 2 for x in values]  
    callback(result)
```

```
def show_result(data):  
    print("Kết quả:", data)
```

```
process_data([1, 2, 3], show_result)
```

Ý nghĩa

- `show_result` là callback.
- `process_data` không cần biết chi tiết cách hiển thị kết quả.
- Điều này làm chương trình linh hoạt hơn.

Ứng dụng: Pipeline xử lý dữ liệu

```
def apply_pipeline(value, functions):  
    for func in functions:  
        value = func(value)  
    return value  
  
def add_two(x):  
    return x + 2  
  
def multiply_three(x):  
    return x * 3  
  
def square(x):  
    return x ** 2  
  
pipeline = [add_two, multiply_three, square]  
  
print(apply_pipeline(2, pipeline))
```

Diễn giải kết quả:

2 → add_two → 4

4 → multiply_three → 12

12 → square → 144

Ứng dụng:

Raw data



Cleaning



Normalization



Feature transformation



Prediction

Bài tập thực hành 2

Bài 1. Viết hàm `apply_n_times`

`apply_n_times(func, value, n)`

Yêu cầu:

- Áp dụng `func` lên `value` đúng `n` lần.
- Thử nghiệm với hàm tăng 1 và hàm bình phương.

Ví dụ: `apply_n_times(lambda x: x + 1, 5, 3)` ---> Kết quả : 8

Bài 2. Data pipeline

Viết hàm nhận list số và list hàm biến đổi; áp dụng lần lượt tất cả các hàm lên từng giá trị.

Ví dụ : input là: `data = [1, 2, 3]`

`functions = [ lambda x: x * 2, lambda x: x + 1 ]`

output là : `[3, 5, 7]`

Closure và wrapper

- Phạm vi biến trong Python: Python tìm biến theo quy tắc **LEGB**
 1. **L (Local)**: Phạm vi cục bộ. Đây là các biến được định nghĩa bên trong một hàm (hoặc hàm lambda) và không được khai báo là toàn cục.
 2. **E (Enclosing)**: Phạm vi bao bọc. Xuất hiện trong các hàm lồng nhau (nested functions). Đây là phạm vi của hàm cha (hàm bên ngoài) đối với hàm con (hàm bên trong).
 3. **G (Global)**: Phạm vi toàn cục. Đây là các biến được định nghĩa ở cấp cao nhất của tệp script/module hoặc được khai báo bằng từ khóa global.
 4. **B (Built-in)**: Phạm vi tích hợp sẵn. Đây là các tên biến/hàm được Python tải sẵn khi khởi động (ví dụ: print, len, int, ValueError).

Phạm vi biến trong Python

Ký hiệu	Phạm vi	Ý nghĩa
L	Local	Bên trong hàm hiện tại
E	Enclosing	Trong hàm bao ngoài
G	Global	Toàn bộ module
B	Built-in	Hàm/tên có sẵn của Python

Phạm vi biến trong Python

```
x = "global"

def outer():
    x = "enclosing"

    def inner():
        x = "local"
        print(x)

    inner()

outer()
```

Kết quả: local

Function closure là gì?

Closure xuất hiện khi:

- Có một hàm lồng bên trong một hàm khác.
- Hàm bên trong sử dụng biến của hàm ngoài.
- Hàm ngoài trả về hàm bên trong.
- Hàm bên trong vẫn “nhớ” biến của hàm ngoài sau khi hàm ngoài kết thúc.

```
def make_multiplier(n):  
    def multiplier(x):  
        return x * n  
  
    return multiplier  
  
multiply_by_2 = make_multiplier(2)
```

Ví dụ closure

```
def make_multiplier(n):  
    def multiplier(x):  
        return x * n  
  
    return multiplier  
  
multiply_by_2 = make_multiplier(2)  
multiply_by_5 = make_multiplier(5)  
  
print(multiply_by_2(10))  
print(multiply_by_5(10))
```

Kết quả:
20
50

Điểm cốt lõi

multiply_by_2 nhớ $n = 2$.
multiply_by_5 nhớ $n = 5$.
Hai closure có trạng thái riêng.

Ví dụ: Closure để tạo hàm kiểm tra ngưỡng

```
def make_threshold_checker(threshold):  
    def check(value):  
        return value > threshold  
  
    return check  
  
is_high_pm25 = make_threshold_checker(25)  
  
print(is_high_pm25(18.2))  
print(is_high_pm25(35.6))
```

Kết quả:
False
True

Ứng dụng:

- Kiểm tra giá trị vượt ngưỡng.
- Thiết lập quy tắc phân loại linh hoạt.
- Tạo hàm được cấu hình sẵn.

Ví dụ: Closure giữ trạng thái

```
def create_counter():  
    count = 0  
  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
  
    return increment  
  
counter = create_counter()  
print(counter())  
print(counter())  
print(counter())
```

Kết quả :

1
2
3

Từ khóa: `nonlocal`

- Dùng khi muốn thay đổi biến thuộc phạm vi hàm bao ngoài.
- Không phải biến local của hàm trong.
- Không phải biến global của module.

Wrapper là gì?

- Wrapper là hàm bao quanh một hàm khác nhằm thêm hành vi trước hoặc sau khi gọi hàm gốc.
- Wrapper có thể dùng để:
 - Ghi log.
 - Đo thời gian thực thi.
 - Kiểm tra quyền truy cập.
 - Kiểm tra dữ liệu đầu vào.
 - Bắt lỗi.
 - Lưu kết quả tạm thời.

```
def wrapper():  
    print("Bắt đầu thực hiện")  
    result = original_function()  
    print("Kết thúc thực hiện")  
    return result
```

Ví dụ wrapper thủ công

```
def greet(name):  
    return f'Hello, {name}!'
```

```
def greet_with_log(name):  
    print("Bắt đầu gọi hàm greet")  
    result = greet(name)  
    print("Đã gọi xong hàm greet")  
    return result
```

```
print(greet_with_log("Nam"))
```

Bắt đầu gọi hàm greet
Đã gọi xong hàm greet
Hello, Nam!

Hạn chế

- Nếu có 20 hàm cần log, ta phải viết 20 wrapper tương tự.
- Code bị lặp.
- Decorator được tạo ra để giải quyết vấn đề tái sử dụng này.

Bài tập thực hành 3

- Bài 1. Closure

Viết hàm: `make_power(power)`

Hàm trả về một hàm tính: $x ** power$

Ví dụ: khi gọi hàm

```
square = make_power(2)
```

```
cube = make_power(3)
```

```
print(square(4))
```

```
print(cube(4))
```

Bài tập thực hành 3

- Bài 2. Closure có trạng thái

Viết hàm : `create_accumulator()`

Mỗi lần gọi hàm được trả về, giá trị mới được cộng dồn vào tổng hiện có.

Ví dụ: khi gọi hàm

```
accumulator = create_accumulator()
```

```
print(accumulator(10))
```

```
print(accumulator(5))
```

```
print(accumulator(20))
```

Decorator

- Decorator là một higher-order function dùng để:
 - Nhận vào một hàm.
 - Tạo hoặc sử dụng wrapper.
 - Trả về một hàm mới có hành vi mở rộng.

```
def decorator(func):  
    def wrapper():  
        print("Before")  
        func()  
        print("After")  
  
    return wrapper
```

Decorator không dùng cú pháp @

```
def greet():  
    print("Hello!")  
  
def simple_decorator(func):  
    def wrapper():  
        print("Before calling function")  
        func()  
        print("After calling function")  
  
    return wrapper  
  
greet = simple_decorator(greet)  
  
greet()
```

Kết quả:
Before calling function
Hello!
After calling function

Function `greet` ban đầu đã được thay thế bằng function wrapper.

Decorator với cú pháp @

```
def simple_decorator(func):  
    def wrapper():  
        print("Before calling function")  
        func()  
        print("After calling function")  
  
    return wrapper  
  
@simple_decorator  
def greet():  
    print("Hello!")  
  
greet()
```

“Ký hiệu @ chỉ là syntax ngắn gọn.”

Tương đương:
greet = simple_decorator(greet)

Decorator có tham số

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f'Calling: {func.__name__}')
        print(f'args: {args}')
        print(f'kwargs: {kwargs}')

        return func(*args, **kwargs)

    return wrapper

@log_call
def add(a, b):
    return a + b

print(add(3, 5))
```

Nếu hàm gốc có tham số, wrapper cần nhận các tham số đó.

add(3, 5)
↓
wrapper(3, 5)
↓
func(3, 5)

Kết quả:
Calling: add
args: (3, 5)
kwargs: {}
8

Ý nghĩa của *args và **kwargs

Cú pháp	Thu thập	Kiểu dữ liệu bên trong hàm
*args	Đôi số vị trí	Tuple
**kwargs	Đôi số có tên	Dictionary

```
def example(*args, **kwargs):  
    print(args)  
    print(kwargs)  
  
example(10, 20, name="An", score=8.5)
```

Kết quả:
(10, 20)
{'name': 'An', 'score': 8.5}

Ví dụ: Decorator đo thời gian thực thi

```
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f'{func.__name__} chạy trong {end - start:.6f} giây')
        return result

    return wrapper

@timer
def calculate_sum(n):
    return sum(range(n))

calculate_sum(10000000000)
```

Start timer
↓
calculate_sum()
↓
End timer
↓
Print execution time

Kết quả :
calculate_sum chạy trong 20.889434 giây

Ví dụ: Decorator ghi log

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f"[LOG] Bắt đầu: {func.__name__}")
        result = func(*args, **kwargs)
        print(f"[LOG] Kết thúc: {func.__name__}")
        return result

    return wrapper

@log_call
def calculate_mean(values):
    return sum(values) / len(values)

print(calculate_mean([10, 20, 30]))
```

[LOG] Bắt đầu



calculate_mean



[LOG] Kết thúc

Không sửa `calculate_mean()` nhưng vẫn thêm được logging

Kết quả:

[LOG] Bắt đầu: calculate_mean

[LOG] Kết thúc: calculate_mean

20.0

Decorator: Vấn đề metadata của hàm

- Khi dùng decorator đơn giản: `print(calculate_mean.__name__)`
có thể cho kết quả: `wrapper`
---> Điều này làm mất tên và docstring của hàm gốc.
- **Giải pháp:** dùng `functools.wraps`

Decorator tốt hơn với functools.wraps

```
from functools import wraps
def log_call(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f'Calling: {func.__name__}')
        return func(*args, **kwargs)

    return wrapper

@log_call
def calculate_mean(values):
    """Tính trung bình của dãy số."""
    return sum(values) / len(values)

print(calculate_mean.__name__)
print(calculate_mean.__doc__)
```

Khuyến nghị:

Khi viết decorator thực tế, nên dùng `@wraps(func)`

`wraps()` giúp wrapper giữ lại metadata của function gốc, chẳng hạn tên và docstring.

Kết quả:

`calculate_mean`

Tính trung bình của dãy số.

Tổng kết Chương 2

- Hàm trong Python là đối tượng hạng nhất.
- Docstring giúp tài liệu hóa code và nâng cao khả năng bảo trì.
- Lambda phù hợp với các biểu thức ngắn, thường dùng cùng map, filter, sorted.
- Higher-order functions nhận hàm làm đối số hoặc trả về hàm.
- Closure cho phép hàm ghi nhớ môi trường bao ngoài.
- Wrapper thêm hành vi xung quanh hàm gốc.
- Decorator là cơ chế tái sử dụng wrapper thông qua cú pháp @.

Bài tập thực hành 4

- Bài 1. Decorator kiểm tra dữ liệu

Viết decorator: `check_non_empty`

Yêu cầu:

- Áp dụng cho hàm nhận list.
- Nếu list rỗng, phát sinh `ValueError`.
- Nếu list không rỗng, gọi hàm gốc bình thường.

Áp dụng cho: `calculate_mean(values)`
`find_max(values)`

Bài tập ôn tập:

Bài toán: Phân tích nhanh dữ liệu PM2.5

```
pm25_values = [12.5, 18.2, 35.1, 28.4, 41.8, 20.6, 30.2]
```

Yêu cầu:

- Viết hàm `calculate_mean(values)` có docstring.
- Dùng `lambda` và `filter()` để lấy giá trị PM2.5 lớn hơn 25.
- Viết closure `make_threshold_checker(threshold)`.
- Viết higher-order function `transform_values(values, func)` (lấy mỗi giá trị pm25 chia cho giá trị lớn nhất của list `pm25_values`)
- Viết decorator `log_call` để in tên hàm mỗi khi hàm được gọi.
- Áp dụng decorator cho `calculate_mean`.